

BLOCO 5 • SISTEMAS OPERACIONAIS DISTRIBUÍDOS

Sistemas de Arquivos Distribuídos

NFS, GFS e o problema da coerência de cache

Fechamento do bloco de sistemas operacionais distribuídos aberto na Aula 24

AGENDA

O que vamos ver hoje

Aula 26 de 66h — fecha o bloco de SO distribuídos aberto na Aula 24.

01

O que é um DFS

A ilusão de um único sistema de arquivos sobre máquinas diferentes

02

Requisitos centrais

Transparência de acesso/localização e semântica de consistência

03

Estudo de caso: NFS

Arquitetura cliente-servidor, protocolo stateless, file handles

04

Estudo de caso: GFS

Master, chunkservers e replicação para arquivos grandes

05

De GFS a HDFS

Como a arquitetura do Google influenciou o Hadoop

06

Caching e coerência

Desempenho no cliente x o problema da consistência

Retomando: do escalonamento aos arquivos

O bloco de SO distribuídos começou olhando para a arquitetura geral (Aula 24) e para como um SOD decide onde cada processo executa (Aula 25). Hoje fechamos o bloco perguntando: e os arquivos que esses processos usam — onde eles ficam, e como vários processos em máquinas diferentes os compartilham?

1

Aula 24 — Arquitetura

SO de Rede x SO Distribuído; modelos workstation, processor pool e híbrido; papel do middleware.

2

Aula 25 — Escalonamento

Gerência de processos e escalonamento distribuído: migração de processos e balanceamento de carga entre nós.

3

Aula 26 — Arquivos (hoje)

Como um SOD estende o sistema de arquivos para além de uma única máquina, com transparência e consistência.

CONCEITO

O que é um sistema de arquivos distribuído?

Um sistema de arquivos distribuído (DFS) permite que processos em execução em máquinas diferentes acessem e compartilhem arquivos como se estivessem todos em um único sistema de arquivos local.

Do ponto de vista de quem programa ou usa o sistema, existe uma única hierarquia de diretórios coerente — mesmo que, por trás dela, os arquivos estejam fisicamente espalhados por vários servidores diferentes.

É a mesma ideia de transparência que já vimos para processamento e comunicação — agora aplicada ao armazenamento.

VISÃO DO CLIENTE

/dados (namespace único)

Camada de sistema de arquivos distribuído

REALIDADE FÍSICA

Servidor A

guarda parte dos arquivos

Servidor B

guarda parte dos arquivos

Servidor C

guarda parte dos arquivos

O cliente nunca precisa saber em qual servidor cada arquivo está — isso é transparência de localização.

Requisitos centrais de um DFS

1

Transparência de acesso e localização

Acesso: as mesmas operações servem para ler ou escrever um arquivo local e um arquivo remoto — o programa cliente não muda.

Localização: o nome do arquivo não revela onde ele está fisicamente armazenado, e o arquivo pode ser movido entre servidores sem que o caminho usado pelo cliente precise mudar.

2

Semântica de consistência

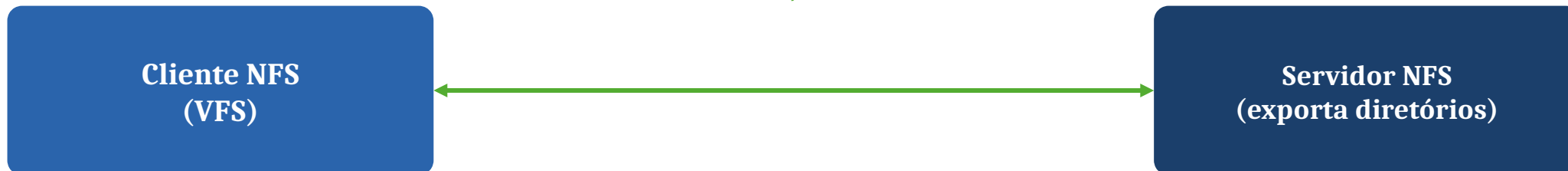
O que um cliente deve enxergar quando outro cliente, em outra máquina, está lendo ou escrevendo o mesmo arquivo ao mesmo tempo?

Garantir que toda escrita apareça imediatamente para todos (semântica Unix) é caro em rede. Por isso muitos DFS relaxam essa garantia — como a semântica de sessão do NFS clássico, em que mudanças só ficam visíveis a outros clientes depois que o arquivo é fechado.

NFS: arquitetura cliente-servidor

O Network File System (NFS), criado pela Sun Microsystems nos anos 1980, é o DFS de referência do modelo cliente-servidor mais simples.

chamadas de procedimento remoto (RPC), tradicionalmente sobre
UDP/TCP



Protocolo stateless

Nas versões originais (NFSv2/v3), o servidor não guarda estado sobre quais arquivos cada cliente tem abertos — cada requisição é autocontida.



File handle

Referência opaca ao arquivo (dispositivo, número de inode, geração), obtida via protocolo de montagem — independe do caminho textual.



Mount protocol

Antes de acessar, o cliente monta um diretório exportado pelo servidor e recebe o file handle da raiz daquela árvore.

Por que o NFS ser stateless importa

Um protocolo stateless simplifica drasticamente a recuperação depois de uma falha: como o servidor não mantém sessão nem estado do que cada cliente estava fazendo, ele pode reiniciar sem reconstruir nada — o cliente simplesmente reenvia a última requisição.

1

Idempotência

Operações são desenhadas para que repetir a mesma requisição não cause efeito colateral — essencial quando o cliente reenvia após timeout.

2

Sem estado de sessão

O servidor não sabe quais arquivos cada cliente tem 'abertos'; cada chamada carrega toda a informação necessária (ex.: file handle + deslocamento).

3

Recuperação simples

Um crash do servidor NFS clássico não perde estado de clientes — ao voltar, ele volta a responder normalmente, sem protocolo de reconciliação.

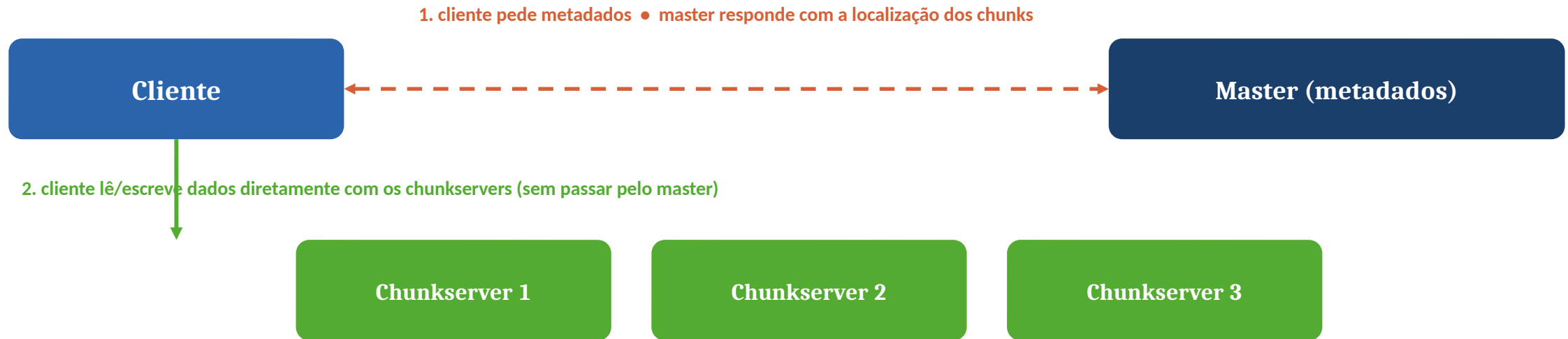
4

Custo: consistência mais fraca

A contrapartida é a semântica de sessão (mudanças só visíveis após o fechamento do arquivo) em vez de consistência imediata.

GFS: master, chunkservers e replicação

O Google File System (Ghemawat, Gobioff e Leung, 2003) foi projetado para arquivos muito grandes, leitura majoritariamente sequencial e escrita majoritariamente por append, rodando sobre milhares de máquinas comuns onde falhas são a norma, não a exceção.



O master nunca fica no caminho dos dados — só resolve metadados. Isso evita que ele vire gargalo.

64 MB

tamanho fixo de cada chunk

3 réplicas

número padrão de cópias por chunk

Append-only

padrão de escrita otimizado pelo projeto

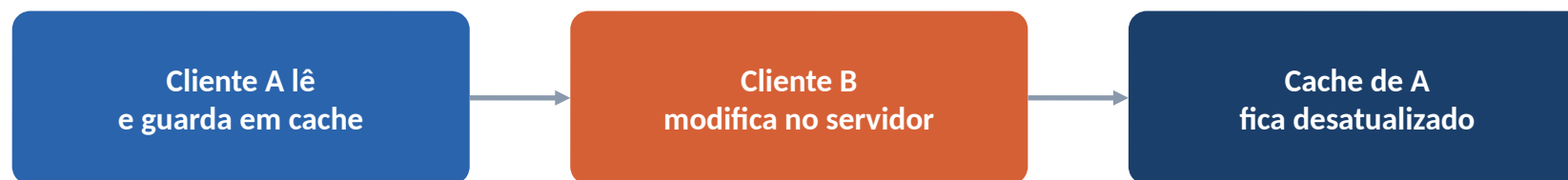
De GFS a HDFS

O artigo original do GFS (SOSP 2003) influenciou diretamente o projeto do HDFS (Hadoop Distributed File System), que reaproveita a mesma divisão de papéis com outros nomes:

Papel	No GFS	No HDFS	Função
Nó de metadados	Master	NameNode	Namespace, mapeamento arquivo → blocos, localização das réplicas.
Nó de dados	Chunkserver	DataNode	Armazena os blocos/chunks de dados propriamente ditos.
Unidade de divisão	Chunk (64 MB)	Block (64 MB originalmente, hoje comumente 128 MB)	Fragmento fixo em que cada arquivo grande é dividido.
Replicação	3 réplicas (padrão)	3 réplicas (padrão, configurável)	Tolerância a falhas de disco e de nó inteiro.
Padrão de escrita	Append-only	Write-once-read-many	Otimizado para grandes lotes de dados que crescem, não para edição.

Caching no cliente e o problema da coerência

Cache no cliente é a técnica central de desempenho em DFS: evita ida à rede a cada leitura, reduz latência percebida e alivia a carga do servidor.



Esse é o mesmo dilema da coerência de cache em hardware, em que vários processadores mantêm cópias da mesma linha de memória — só que aqui, em vez de processadores, são clientes espalhados pela rede, com latência muito maior.

Validação por timestamp

O cliente confirma com o servidor, antes de usar a cópia em cache, se ela ainda é a versão mais recente.

Invalidação por callback

O servidor avisa proativamente os clientes com cópias em cache quando o arquivo muda (usado em sistemas como o AFS).

Semântica de sessão

O NFS clássico adia o problema: mudanças só se tornam visíveis para outros clientes depois que o arquivo é fechado.

PRÓXIMA AULA

Aula 27 — Revisão Geral do 2º Bimestre

e orientação do trabalho final

Com o bloco de SO distribuídos fechado — arquitetura, escalonamento e agora arquivos distribuídos — a próxima aula retoma os principais conceitos do 2º bimestre e orienta os checkpoints e a entrega final do trabalho evolutivo.

Traga dúvidas sobre consenso, replicação, tolerância a falhas e SO distribuídos para a revisão.

REFERÊNCIAS

Bibliografia desta aula



TANENBAUM, Andrew S.; VAN STEEN, Maarten.

Distributed Systems: Principles and Paradigms (ou edição atual Distributed Systems). Pearson. — Capítulo sobre sistemas de arquivos distribuídos: NFS, transparência e semântica de consistência.



COULOURIS, George et al.

Sistemas Distribuídos: Conceitos e Projeto. 5ª ed. Porto Alegre: Bookman. — Capítulo sobre serviços de arquivos distribuídos: requisitos, estudos de caso e caching.



GHEMAWAT, Sanjay; GOBIOFF, Howard; LEUNG, Shun-Tak.

"The Google File System". In: Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP), 2003. — Artigo original que descreve a arquitetura do GFS.